

Olist Snowflake Portfolio Project — Reference Document

Dataset: Brazilian E-Commerce Public Dataset by Olist (Kaggle) **Certification context:** SnowPro Core certified, building a strong portfolio project

Project Ideas Considered

Idea #1 — End-to-end analytics engineering pipeline SELECTED

Build a full modern data stack: raw file ingestion → Snowflake staging → dbt transformations → semantic layer → Power BI dashboard.

Why it was chosen: Maps directly to how real data teams work. Covers the full cloud data stack in one cohesive project. Most impressive for a Data Engineering profile.

Key components:

- AWS S3 as raw data landing zone
 - Snowflake external stage + Snowpipe for ingestion
 - dbt Core for Bronze → Silver → Gold transformations (medallion architecture)
 - Snowflake Streams + Tasks for automation
 - Power BI via DirectQuery for reporting
-

Idea #2 — Customer segmentation & RFM analysis

Build RFM (Recency, Frequency, Monetary) scoring entirely in Snowflake SQL. Cluster customers into actionable segments. Use Snowpark or dbt for cohort analysis.

Best suited for: Data Analytics / Business Analytics profiles.

Key components: Window functions, CTEs, cohort analysis, Snowpark ML.

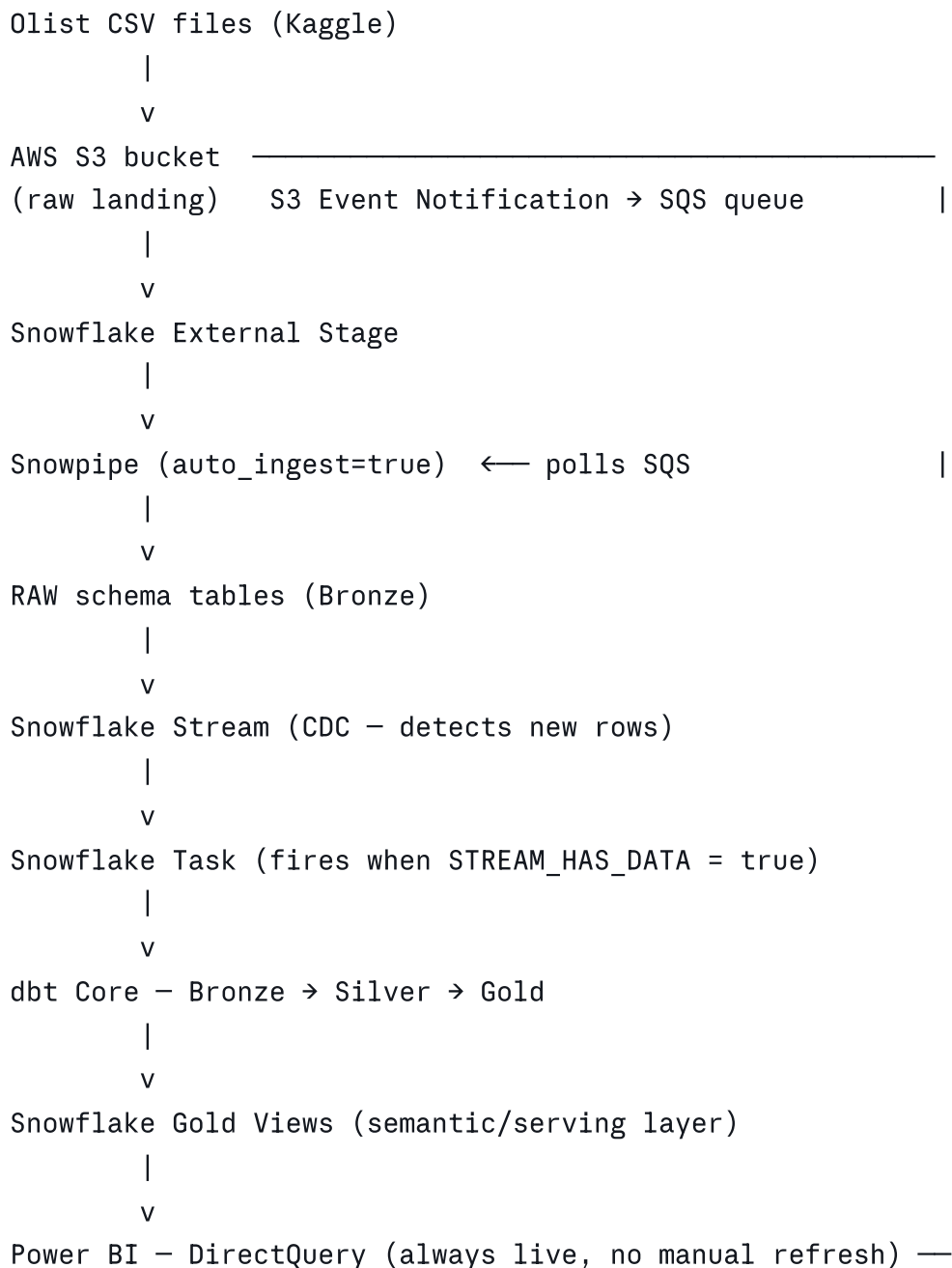
Idea #3 — Review sentiment analysis with Snowflake Cortex AI

Use Snowflake Cortex AI's native `SENTIMENT()` and `TRANSLATE()` functions on the 100k+ Portuguese customer reviews. Correlate sentiment scores with ratings, delivery speed, and product categories. No external ML tools needed.

Best suited for: Data Analytics profile, or as an add-on module to Idea #1.

Key differentiator: Reviews are in Portuguese, so showcasing `TRANSLATE()` + `SENTIMENT()` is genuinely rare in candidate portfolios. Very few people have hands-on Cortex AI experience yet.

Finalized Architecture (Idea #1)



AWS S3 role in the project

- Acts as the raw data landing zone, simulating how real companies store source files before loading to a warehouse
- S3 Event Notifications push to SQS queue automatically on file upload
- Snowpipe polls SQS and triggers `COPY INTO` — fully event-driven, no manual runs
- Skill demonstrated: external stages, IAM role config, `COPY INTO` with `METADATA$FILENAME`

Power BI role in the project

- Connects to Snowflake Gold layer views via the native Snowflake connector

- Use **DirectQuery mode** (not Import mode) — queries Snowflake live, no manual refresh needed
 - Power BI always points at Gold views, not raw tables — proves a proper semantic layer was built
 - Dashboard KPIs reflect new data within minutes of a file being dropped in S3
-

Real-Time Pipeline Design

How it works end-to-end

1. CSV file uploaded to S3
2. S3 fires an event notification to an AWS SQS queue automatically
3. Snowpipe polls SQS, detects the new file, runs `COPY INTO` → appends rows to RAW table
4. Snowflake Stream detects new rows (CDC)
5. Snowflake Task fires dbt transformation when `SYSTEM$STREAM_HAS_DATA()` = true
6. Bronze → Silver → Gold layers rebuilt automatically
7. Power BI DirectQuery picks up changes on next report load — no manual refresh

Simulating real-time with the Olist static dataset

- Split `olist_orders_dataset.csv` into 12 quarterly files by `order_purchase_timestamp`
 - Upload files one by one in chronological order with a short pause between each
 - Each upload triggers the full pipeline automatically
 - Resume framing: *"pipeline processed 12 incremental data files end-to-end with no manual intervention"*
-

Dataset Structure

Transactional data (event-driven — feed through Snowpipe incrementally)

Table	Description	Load strategy
olist_orders_dataset	Core transaction record, one row per order with timestamps	Snowpipe + Stream
olist_order_items_dataset	Line items within each order, product + seller info	Snowpipe + Stream
olist_order_payments_dataset	Payment method per order, supports multiple payments per order	Snowpipe + Stream

Table	Description	Load strategy
olist_order_reviews_dataset	Customer review after delivery, has own timestamps	Snowpipe + Stream

Master / reference data (static — one-time COPY INTO load)

Table	Description	Load strategy
olist_customers_dataset	Customer records with location info	COPY INTO, one-time
olist_sellers_dataset	Seller records with location info	COPY INTO, one-time
olist_products_dataset	Product catalog with category and dimensions	COPY INTO, one-time
olist_geolocation_dataset	Zip code to lat/long mapping	COPY INTO, one-time
product_category_name_translation	Portuguese category names → English	COPY INTO, one-time

Important nuance — olist_customers_dataset: A customer record is created per order, not per person. The same physical customer placing two orders gets two different `customer_id` values. This means it behaves more like a transactional dimension than a true master record. Worth noting in project documentation — shows you read the data carefully.

Quarterly File Split Strategy

Data distribution

- **2016:** Q3, Q4 only — very small, a few hundred rows each (dataset starts mid-2016)
- **2017:** Q1-Q4 — growing, mid-sized files
- **2018:** Q1-Q4 — largest files, bulk of the data
- **Total: 12 files**

Naming convention

```
s3://olist-project/raw/orders/orders_2016_q3.csv
s3://olist-project/raw/orders/orders_2016_q4.csv
s3://olist-project/raw/orders/orders_2017_q1.csv
```

```
...
s3://olist-project/raw/orders/orders_2018_q4.csv
```

Python split script

```
python

import pandas as pd

df = pd.read_csv('olist_orders_dataset.csv', parse_dates=['order_purchase_timestamp'])
df['year'] = df['order_purchase_timestamp'].dt.year
df['quarter'] = df['order_purchase_timestamp'].dt.quarter

for (year, quarter), group in df.groupby(['year', 'quarter']):
    filename = f'orders_{year}_q{quarter}.csv'
    group.drop(columns=['year', 'quarter']).to_csv(filename, index=False)
    print(f'{filename}: {len(group)} rows')
```

Metadata columns to add to RAW table

```
sql

CREATE TABLE raw.orders (
  order_id          VARCHAR,
  customer_id       VARCHAR,
  order_status      VARCHAR,
  order_purchase_timestamp TIMESTAMP,
  -- ... other columns from CSV ...
  _source_file      VARCHAR,          -- populated via METADATA$FILENAME
  _loaded_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP()
);
```

Adding `_source_file` and `_loaded_at` makes the pipeline auditable — every row is traceable to its source file and load time. This is a professional best practice worth highlighting on the resume.

Profile Fit Summary

Profile	Best idea	Framing tip
Data Engineering	Idea #1 (this project)	Lead with pipeline mechanics: Snowpipe, Streams, Tasks, dbt, medallion architecture

Profile	Best idea	Framing tip
Data Analytics	Idea #2 (RFM) or Idea #3 (Sentiment)	Lead with technique: window functions, RFM scoring, cohort analysis
Business Analytics	Idea #2 (RFM) or seller dashboard	Lead with business question: which sellers churn, which segments drive revenue

Next Steps (not yet discussed)

- ☐ Gold layer star schema design
- ☐ Full Snowpipe + Stream + Task SQL setup
- ☐ dbt model structure (sources, staging, marts)
- ☐ Power BI dashboard KPIs and report pages
- ☐ GitHub repo structure and README

Document covers conversation up to: dataset structure, project selection, architecture decisions, real-time pipeline design, and quarterly file split strategy.